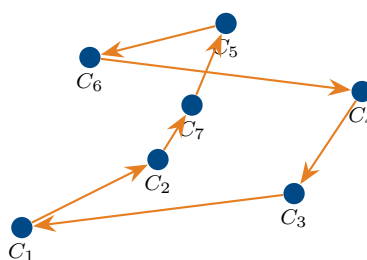


Algorithme Génétique pour le Problème du Voyageur de Commerce

Documentation Technique Détaillée



Résumé

Ce document présente une implémentation complète d'un algorithme génétique pour résoudre le problème du voyageur de commerce (TSP) bi-objectif. L'algorithme optimise simultanément la distance totale et le coût total du parcours à travers 15 villes.

December 31, 2025

Contents

1	Introduction et Formulation du Problème	2
1.1	Description du Problème	2
1.2	Formulation Mathématique	2
2	Analyse Combinatoire de l'Espace de Recherche	3
2.1	Comptage Naïf : $n!$ Permutations	3
2.2	Source d'Équivalence 1 : Symétrie Rotationnelle	3
2.3	Source d'Équivalence 2 : Symétrie Directionnelle (TSP Symétrique)	4
2.4	Taille Réelle de l'Espace de Recherche	5
2.5	Nombre de Représentations par Tour	5
2.6	Implications pour l'Algorithme Génétique	6
3	Structure des Données et Représentation	6
3.1	Chargement des Données	7
3.2	Représentation d'une Solution	8
4	Fonction d'Évaluation (Fitness)	10
4.1	Principe de l'Évaluation	10
4.2	Implémentation de l'Évaluation	10
5	Paramètres de l'Algorithme Génétique	12
5.1	Taille de la Population	12
5.2	Taux de Croisement : 70%	13
5.2.1	Rôle du Croisement	13
5.2.2	Justification du Taux de 70%	13
5.3	Taux de Mutation : 15%	13
5.3.1	Rôle Fondamental de la Mutation	13
5.3.2	Justification du Taux de 15%	14
5.4	Taille du Tournoi : 5	14
5.4.1	Contrôle de la Pression de Sélection	14
5.4.2	Analyse Probabiliste du Tournoi de Taille 5	15
5.5	Interaction entre les Paramètres	15
6	Opérateur de Sélection	15
6.1	Sélection par Tournoi	15
6.2	Avantages de la Sélection par Tournoi	17

7	Opérateur de Croisement	17
7.1	Croisement à Deux Points	17
7.2	Illustration du Croisement	17
7.3	Correction des Doublons	18
8	Opérateur de Mutation	19
8.1	Mutation par Échange	19
8.2	Rôle de la Mutation	20
9	Algorithme Génétique Principal	20
9.1	Vue d'Ensemble	20
9.2	Création de la Population Initiale	21
9.3	Création d'une Nouvelle Génération	21
9.4	Boucle Principale de l'Algorithme	22
10	Sauvegarde et Export des Résultats	24
10.1	Format XML de Sortie	24
10.2	Logique de Sauvegarde	25
11	Conclusion et Perspectives	25
11.1	Récapitulatif	25
11.2	Améliorations Possibles	26

1 Introduction et Formulation du Problème

1.1 Description du Problème

Le problème du voyageur de commerce (Traveling Salesman Problem, TSP) est l'un des problèmes d'optimisation combinatoire les plus célèbres et les plus étudiés en informatique théorique et en recherche opérationnelle. Ce problème appartient à la classe des problèmes NP-difficiles, ce qui signifie qu'aucun algorithme polynomial n'est connu pour le résoudre de manière exacte.

Définition formelle du TSP

Soit un ensemble de n villes $V = \{C_1, C_2, \dots, C_n\}$ et une matrice de distances $D = (d_{ij})$ où d_{ij} représente la distance entre les villes C_i et C_j . Le problème consiste à trouver un **circuit hamiltonien** (un cycle passant par chaque ville exactement une fois et revenant au point de départ) qui minimise la distance totale parcourue.

Dans notre implémentation, nous considérons un ensemble de $n = 15$ villes. Le voyageur doit partir d'une ville de départ, visiter chaque ville exactement une seule fois, puis revenir à la ville de départ. Ce qui distingue notre formulation est son caractère **bi-objectif**: chaque déplacement entre deux villes C_i et C_j est caractérisé non seulement par une distance d_{ij} , mais également par un coût c_{ij} .

1.2 Formulation Mathématique

Pour une solution (un parcours) $S = [C_{i_1} \rightarrow C_{i_2} \rightarrow \dots \rightarrow C_{i_n} \rightarrow C_{i_1}]$, nous définissons deux fonctions objectif:

$$f_1(S) = \sum_{k=1}^{n-1} d_{i_k, i_{k+1}} + d_{i_n, i_1} \quad (\text{Distance totale}) \quad (1)$$

$$f_2(S) = \sum_{k=1}^{n-1} c_{i_k, i_{k+1}} + c_{i_n, i_1} \quad (\text{Coût total}) \quad (2)$$

La fonction objectif combinée que l'algorithme cherche à minimiser est définie comme la somme pondérée (avec des poids égaux) des deux objectifs:

$$F(S) = f_1(S) + f_2(S) \quad (3)$$

Contraintes du problème

Le problème est soumis aux contraintes suivantes:

1. Chaque ville doit être visitée exactement une fois
2. Le voyageur doit revenir à la ville de départ (circuit fermé)
3. Les sous-tournées sont interdites (le parcours doit former un seul cycle)

2 Analyse Combinatoire de l'Espace de Recherche

Une compréhension précise de la taille de l'espace de recherche est essentielle pour apprécier la complexité du problème TSP et justifier l'utilisation de métaheuristiques comme les algorithmes génétiques. Cette section analyse rigoureusement le nombre de solutions distinctes en tenant compte des symétries du problème.

2.1 Comptage Naïf : $n!$ Permutations

À première vue, avec n villes, on pourrait penser qu'il existe $n!$ parcours possibles, correspondant aux $n!$ permutations des villes. Pour $n = 15$ villes:

$$15! = 1\,307\,674\,368\,000 \approx 1.3 \times 10^{12}$$

Cependant, ce comptage surestime considérablement le nombre de **tours physiquement distincts** en raison de deux sources d'équivalence que nous allons analyser.

2.2 Source d'Équivalence 1 : Symétrie Rotationnelle

Un tour TSP est un **cycle**, pas une séquence linéaire. Le point de départ est arbitraire car le voyageur revient à son point de départ après avoir complété le circuit. Considérons le tour suivant avec 4 villes:

Les quatre représentations suivantes décrivent **exactement le même parcours physique**:

$$C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4 \rightarrow C_1$$

$$C_2 \rightarrow C_3 \rightarrow C_4 \rightarrow C_1 \rightarrow C_2$$

$$C_3 \rightarrow C_4 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3$$

$$C_4 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4$$

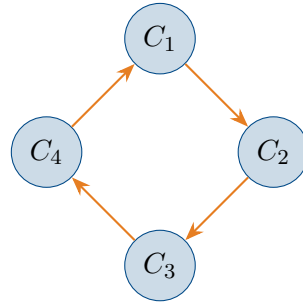


Figure 1: Un tour cyclique entre 4 villes

Ces quatre séquences ne sont que des **rotations** du même cycle. Pour un tour de n villes, il existe exactement n rotations équivalentes. On peut donc fixer arbitrairement la ville de départ (par exemple, toujours commencer par C_1), ce qui élimine ce facteur de redondance.

Réduction par symétrie rotationnelle

En fixant la ville de départ, le nombre de permutations distinctes passe de $n!$ à :

$$\frac{n!}{n} = (n - 1)!$$

2.3 Source d'Équivalence 2 : Symétrie Directionnelle (TSP Symétrique)

La seconde source d'équivalence n'existe que dans le cas du **TSP symétrique**, où les matrices de distances et de coûts satisfont :

$$d_{ij} = d_{ji} \quad \text{et} \quad c_{ij} = c_{ji} \quad \forall i, j$$

Dans ce cas, parcourir un tour dans le sens horaire ou anti-horaire donne exactement le même coût total. Les deux tours suivants sont donc équivalents :

Sens direct: $C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4 \rightarrow C_1$

Sens inverse: $C_1 \rightarrow C_4 \rightarrow C_3 \rightarrow C_2 \rightarrow C_1$

Réduction par symétrie directionnelle

Pour un TSP symétrique, chaque tour physique a exactement 2 représentations (sens direct et sens inverse). Le nombre de tours distincts est donc divisé par 2.

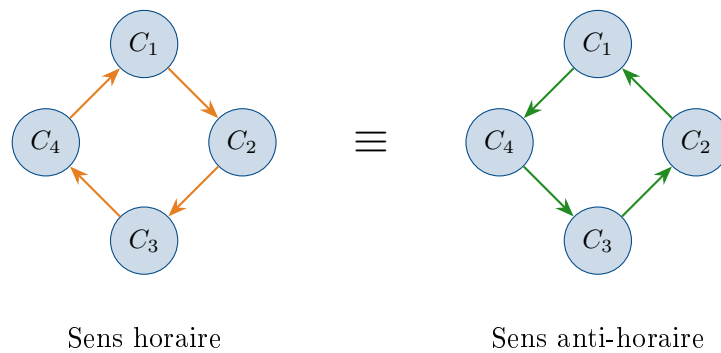


Figure 2: Équivalence directionnelle: même tour parcouru dans les deux sens

2.4 Taille Réelle de l'Espace de Recherche

En combinant les deux sources de symétrie, nous obtenons le nombre exact de tours physiquement distincts:

Espace de recherche du TSP

TSP Asymétrique (matrices non symétriques):

$$\text{Nombre de tours distincts} = (n - 1)!$$

TSP Symétrique (matrices symétriques):

$$\text{Nombre de tours distincts} = \frac{(n - 1)!}{2}$$

Pour notre problème avec $n = 15$ villes:

 Table 1: Comparaison des espaces de recherche pour $n = 15$ villes

Type de comptage	Formule	Valeur numérique
Permutations brutes	$n!$	1 307 674 368 000
Après fixation du départ	$(n - 1)!$	87 178 291 200
TSP Symétrique (final)	$\frac{(n - 1)!}{2}$	43 589 145 600

2.5 Nombre de Représentations par Tour

Une façon équivalente de comprendre ces symétries est de compter combien de représentations (permutations) correspondent à un même tour physique:

Représentations équivalentes

Dans un TSP symétrique avec n villes, chaque tour physique unique possède exactement:

$$n \times 2 = 2n \text{ représentations équivalentes}$$

où n provient des rotations et 2 provient des deux sens de parcours.

Pour $n = 15$: chaque tour a **30** représentations équivalentes.

On vérifie la cohérence: $\frac{15!}{30} = \frac{1\,307\,674\,368\,000}{30} = 43\,589\,145\,600 = \frac{14!}{2} \checkmark$

2.6 Implications pour l'Algorithme Génétique

Cette analyse a des conséquences pratiques importantes pour l'implémentation de l'algorithme génétique. Stocker des solutions équivalentes gaspille de la mémoire et fausse les statistiques de diversité. Pour éviter cela, on utilise une **forme canonique**: chaque tour est représenté par un unique représentant de sa classe d'équivalence.

La canonicalisation consiste à fixer la ville de départ (élimination des rotations) et à imposer une convention sur le sens du parcours (élimination de la symétrie directionnelle). Par exemple, on peut exiger que le deuxième élément du parcours ait un indice plus petit que le dernier élément.

Canonicalisation d'un tour

Tour original: $[C_5, C_2, C_8, C_3, C_1]$

Étape 1 (Rotation): Placer la ville d'indice minimal en tête $\Rightarrow [C_1, C_5, C_2, C_8, C_3]$

Étape 2 (Direction): Comparer indices du 2e et dernier élément.

- 2e élément: C_5 (indice 5)
- Dernier élément: C_3 (indice 3)
- Comme $5 > 3$, on inverse: $[C_1, C_3, C_8, C_2, C_5]$

Forme canonique finale: $[C_1, C_3, C_8, C_2, C_5]$

3 Structure des Données et Représentation

3.1 Chargement des Données

Les données du problème sont stockées dans deux fichiers CSV contenant respectivement la matrice des distances et la matrice des coûts. Chaque matrice est de dimension 15×15 , représentant les liaisons entre toutes les paires de villes.

```
1 def lire_matrice_csv(chemin_fichier):
2     """
3     Lit une matrice depuis un fichier CSV.
4
5     La premiere ligne contient les noms des villes (C1, C2,
6     ..., C15).
7     Chaque ligne suivante contient les valeurs numeriques.
8     """
9     matrice = []
10    villes = []
11
12    with open(chemin_fichier, 'r', encoding='utf-8') as fichier:
13        lecteur = csv.reader(fichier)
14
15        # Premiere ligne = en-tete avec noms des villes
16        entete = next(lecteur)
17        villes = entete[1:] # Ignorer la premiere colonne
18
19        # Lire les lignes de donnees
20        for ligne in lecteur:
21            if ligne:
22                valeurs = [int(v) for v in ligne[1:]]
23                matrice.append(valeurs)
24
25    return matrice, villes
```

Listing 1: Fonction de lecture d'une matrice CSV

Les données sont ensuite regroupées dans un dictionnaire central qui servira de référence tout au long de l'exécution de l'algorithme:

```
1 def charger_donnees(fichier_distances, fichier_couts):
2     """
3     Charge les donnees completes du probleme TSP.
4     Retourne un dictionnaire contenant les villes et les deux
```

```

    matrices.
5     """
6     distances, villes = lire_matrice_csv(fichier_distances)
7     couts, _ = lire_matrice_csv(fichier_couts)
8
9     donnees = {
10         'villes': villes,          # Liste: ['C1', 'C2', ...,
                                     # 'C15']
11         'distances': distances,    # Matrice 15x15 des distances
12         'couts': couts             # Matrice 15x15 des couts
13     }
14
15     return donnees

```

Listing 2: Chargement complet des données

3.2 Représentation d'une Solution

Une solution dans notre algorithme génétique est représentée par un dictionnaire Python contenant quatre champs essentiels. Cette structure de données encapsule à la fois le génotype (le parcours) et le phénotype (les valeurs de fitness) de chaque individu.

Table 2: Structure d'une solution

Clé	Type	Description
parcours	Liste	Permutation des villes représentant l'ordre de visite $[C_1, C_4, C_7, \dots]$
distance	Entier	Distance totale du parcours $f_1(S)$
cout	Entier	Coût total du parcours $f_2(S)$
score	Entier	Valeur de la fonction objectif $F = f_1 + f_2$

Exemple

Une solution typique pourrait ressembler à:

```
solution = {  
    'parcours': ['C1', 'C4', 'C7', 'C10', 'C13', 'C15',  
                'C14', 'C12', 'C11', 'C9', 'C8', 'C6',  
                'C5', 'C3', 'C2'],  
    'distance': 242,  
    'cout': 301,  
    'score': 543  
}
```

Ce parcours signifie: $C_1 \rightarrow C_4 \rightarrow C_7 \rightarrow \dots \rightarrow C_2 \rightarrow C_1$

Les fonctions de création et de manipulation des solutions sont présentées ci-dessous:

```
1 def creer_solution_vide():  
2     """Cree une solution vide (sans parcours defini)."""  
3     return {  
4         'parcours': [],  
5         'distance': 0,  
6         'cout': 0,  
7         'score': 0  
8     }  
9  
10 def creer_solution_aleatoire(donnees):  
11     """  
12     Genere une solution aleatoire en melangeant  
13     aleatoirement la liste des villes.  
14     """  
15     parcours = donnees['villes'].copy()  
16     random.shuffle(parcours)  
17  
18     return {  
19         'parcours': parcours,  
20         'distance': 0,  
21         'cout': 0,  
22         'score': 0  
23     }  
24
```

```

25 def copier_solution(solution):
26     """ Cree une copie independante d'une solution. """
27     return {
28         'parcours': solution['parcours'].copy(),
29         'distance': solution['distance'],
30         'cout': solution['cout'],
31         'score': solution['score']
32     }

```

Listing 3: Fonctions de création de solutions

4 Fonction d'Évaluation (Fitness)

4.1 Principe de l'Évaluation

La fonction d'évaluation, également appelée fonction de *fitness*, est au cœur de tout algorithme génétique. Elle quantifie la qualité de chaque solution et guide le processus de sélection vers des solutions de meilleure qualité.

Pour évaluer une solution $S = [C_{i_1}, C_{i_2}, \dots, C_{i_n}]$, nous calculons la somme des distances et des coûts pour chaque arc du parcours, en n'oubliant pas le retour à la ville de départ:

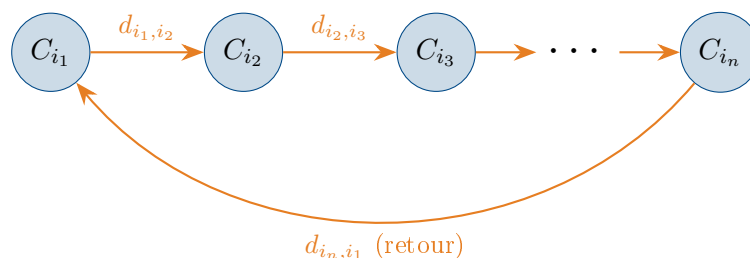


Figure 3: Schéma du calcul de la distance totale d'un parcours

4.2 Implémentation de l'Évaluation

```

1 def evaluer_solution(solution, donnees):
2     """
3     Calcule les valeurs f1 (distance), f2 (cout) et F (score)
4     d'une solution. Cette fonction modifie la solution en place.
5     """
6     parcours = solution['parcours']
7     distance_totale = 0

```

```
8     cout_total = 0
9
10    # Calculer pour chaque etape du parcours (ville i vers
      ville i+1)
11    for i in range(len(parcours) - 1):
12        ville_depart = parcours[i]
13        ville_arrivee = parcours[i + 1]
14
15        distance_totale += obtenir_distance(ville_depart,
16                                            ville_arrivee,
17                                            donnees)
18        cout_total += obtenir_cout(ville_depart,
19                                  ville_arrivee, donnees)
20
21    # IMPORTANT: Ajouter le retour a la ville de depart
22    derniere_ville = parcours[-1]
23    premiere_ville = parcours[0]
24    distance_totale += obtenir_distance(derniere_ville,
25                                      premiere_ville, donnees)
26    cout_total += obtenir_cout(derniere_ville,
27                              premiere_ville, donnees)
28
29    # Mettre a jour la solution
30    solution['distance'] = distance_totale      # f1
31    solution['cout'] = cout_total              # f2
32    solution['score'] = distance_totale + cout_total # F = f1
      + f2
```

Listing 4: Fonction d'évaluation d'une solution

Point crucial

Il est essentiel de ne pas oublier le **retour à la ville de départ**. Le TSP exige un circuit hamiltonien complet, ce qui signifie que le voyageur doit revenir à son point de départ. Omettre ce dernier arc conduirait à une sous-estimation systématique du coût réel de chaque solution.

5 Paramètres de l'Algorithme Génétique

Le choix des paramètres d'un algorithme génétique est crucial pour son efficacité. Ces paramètres contrôlent l'équilibre entre **exploration** (recherche de nouvelles régions de l'espace des solutions) et **exploitation** (amélioration des solutions prometteuses déjà trouvées).

Table 3: Paramètres de l'algorithme génétique

Paramètre	Valeur	Description
taille_population	300	Nombre d'individus (solutions) par génération
nb_generations	500	Nombre maximal d'itérations de l'algorithme
taux_croisement	0.70	Probabilité d'effectuer un croisement (70%)
taux_mutation	0.15	Probabilité d'effectuer une mutation (15%)
taille_tournoi	5	Nombre de participants pour la sélection par tournoi

```

1 PARAMS = {
2     'taille_population': 300,      # 300 solutions aleatoires
3     'nb_generations': 500,        # 500 iterations
4     'taux_croisement': 0.70,     # 70% de croisements
5     'taux_mutation': 0.15,       # 15% de mutations
6     'taille_tournoi': 5           # Selection par tournoi de 5
7 }
```

Listing 5: Définition des paramètres

5.1 Taille de la Population

Une **population de 300 individus** offre un bon équilibre entre diversité génétique et temps de calcul. Avec 15 villes et un TSP symétrique, l'espace des solutions contient $\frac{14!}{2} \approx 43.6$ milliards de tours distincts. Une population de 300 individus ne représente qu'une infime fraction de cet espace ($\approx 7 \times 10^{-9}\%$), mais elle est suffisante pour maintenir une diversité génétique permettant à l'algorithme d'explorer efficacement différentes régions.

5.2 Taux de Croisement : 70%

Le taux de croisement de 0.70 signifie que lors de la création de chaque nouvel individu, il y a 70% de probabilité d'effectuer un croisement entre deux parents sélectionnés, et 30% de probabilité de simplement copier un parent.

5.2.1 Rôle du Croisement

Le croisement est le mécanisme principal de **combinaison des caractéristiques** provenant de différentes solutions. L'intuition est que si un parent possède une bonne séquence de villes dans une région du parcours et qu'un autre parent possède une bonne séquence dans une autre région, le croisement pourrait combiner ces deux caractéristiques favorables dans un descendant encore meilleur.

5.2.2 Justification du Taux de 70%

Ce taux modérément élevé représente un compromis empiriquement validé. Un taux trop élevé (95-100%) serait trop disruptif, car les bonnes solutions seraient constamment fragmentées avant d'être pleinement exploitées, rendant l'algorithme trop exploratoire et ralentissant la convergence. À l'inverse, un taux trop bas (20-30%) transformerait l'algorithme en une recherche quasi-aléatoire avec sélection, car le matériel génétique des différentes bonnes solutions ne serait pas efficacement combiné, rendant la recherche trop locale.

Le taux de 70% permet une recombinaison génétique substantielle tout en préservant certaines bonnes solutions intactes (les 30% copiés). Pour le TSP spécifiquement, le croisement est particulièrement important car un bon tour contient souvent plusieurs sous-séquences de villes proches qui devraient probablement être visitées consécutivement, et le croisement aide à propager ces sous-séquences utiles dans la population.

5.3 Taux de Mutation : 15%

Le taux de mutation de 0.15 signifie que chaque nouvel individu (après croisement ou copie) a 15% de chances de subir une mutation par échange de deux villes.

5.3.1 Rôle Fondamental de la Mutation

La mutation sert un objectif fondamentalement différent du croisement. Alors que le croisement recombine le matériel génétique existant, la mutation **introduit du nouveau matériel génétique** qui n'était présent chez aucun des parents. C'est la protection de l'algorithme contre le piègeage dans des optima locaux.

5.3.2 Justification du Taux de 15%

Ce taux est relativement élevé comparé aux recommandations classiques (1-5% par gène dans les manuels). Cependant, une distinction importante s'impose: dans notre implémentation, la mutation est appliquée **par individu**, non par gène. Chaque individu subit soit une mutation par échange (15% des cas), soit aucune mutation (85% des cas).

Pour le TSP, ce taux relativement élevé se justifie par plusieurs facteurs. Premièrement, l'espace de recherche est énorme ($\frac{14!}{2} \approx 43.6$ milliards de solutions), ce qui nécessite une capacité d'exploration significative. Deuxièmement, un échange de deux villes est une perturbation relativement faible, car il ne modifie que 4 arêtes dans le tour sur les 15 existantes. Troisièmement, le paysage du TSP présente de nombreux optima locaux, rendant la capacité d'en échapper cruciale.

La mutation peut être vue comme l'injection de "bruit" pour prévenir la stagnation. Le taux de 15% injecte suffisamment d'aléatoire pour maintenir la diversité sans submerger la pression de sélection vers les meilleures solutions. Un taux trop élevé (50%+) détruirait la "mémoire" de la population en perturbant constamment les bonnes solutions, transformant la recherche en processus quasi-aléatoire. Un taux trop faible (1-2%) conduirait à une convergence prématurée où la population deviendrait génétiquement homogène, piégeant l'algorithme dans un optimum local sans possibilité d'en sortir.

5.4 Taille du Tournoi : 5

La sélection par tournoi fonctionne en sélectionnant aléatoirement k individus de la population, puis en choisissant le meilleur parmi eux. Avec $k = 5$, chaque sélection implique une mini-compétition entre 5 individus choisis au hasard.

5.4.1 Contrôle de la Pression de Sélection

La taille du tournoi contrôle la **pression de sélection**, c'est-à-dire la force avec laquelle l'algorithme favorise les meilleures solutions par rapport aux moins bonnes.

Un petit tournoi ($k = 2$, appelé "tournoi binaire") représente une pression de sélection faible où même les solutions médiocres ont une chance raisonnable d'être sélectionnées (il suffit de battre un seul autre individu aléatoire). Cela préserve la diversité mais ralentit la convergence.

Un grand tournoi ($k \geq 20$) représente une pression de sélection forte où seules les meilleures solutions sont systématiquement sélectionnées. Cela accélère la convergence mais risque une convergence prématurée vers un optimum local en détruisant rapidement la diversité génétique.

5.4.2 Analyse Probabiliste du Tournoi de Taille 5

Avec une population de 300 individus et un tournoi de taille 5, la probabilité que le meilleur individu de la population soit présent dans un tournoi donné est:

$$P(\text{meilleur dans tournoi}) = 1 - \frac{\binom{299}{5}}{\binom{300}{5}} \approx 1.7\%$$

Cette probabilité peut sembler faible, mais sur les centaines de sélections nécessaires pour créer une nouvelle génération, le meilleur individu sera inclus dans de nombreux tournois. Parallèlement, un individu classé dans les 10% inférieurs peut encore occasionnellement gagner un tournoi (si les 4 autres participants sont encore moins bons), ce qui maintient la diversité génétique.

5.5 Interaction entre les Paramètres

Ces trois paramètres n'opèrent pas de manière isolée mais forment un système interconnecté dont l'équilibre détermine le comportement global de l'algorithme:

Table 4: Effets des combinaisons de paramètres

Combinaison			Comportement
Croisement élevé	+	Mutation faible	Emphase sur la recombinaison des solutions existantes
Croisement faible	+	Mutation élevée	Emphase sur la recherche aléatoire avec sélection
Grand tournoi	+	Mutation faible	Convergence rapide mais risque de piègeage local
Petit tournoi	+	Mutation élevée	Exploration maximale mais convergence lente

La combinaison choisie (70% croisement, 15% mutation, tournoi de 5) représente une approche équilibrée qui combine activement les bonnes solutions partielles, maintient suffisamment de diversité pour échapper aux optima locaux, et applique une pression de sélection régulière mais non écrasante.

6 Opérateur de Sélection

6.1 Sélection par Tournoi

La sélection par tournoi est une méthode de sélection populaire dans les algorithmes génétiques en raison de sa simplicité d'implémentation et de sa capacité à contrôler la

pression de sélection via la taille du tournoi.

Principe de la sélection par tournoi

1. Sélectionner aléatoirement k individus de la population (où k est la taille du tournoi)
2. Comparer ces k individus selon leur valeur de fitness
3. Retourner le meilleur individu parmi les k participants

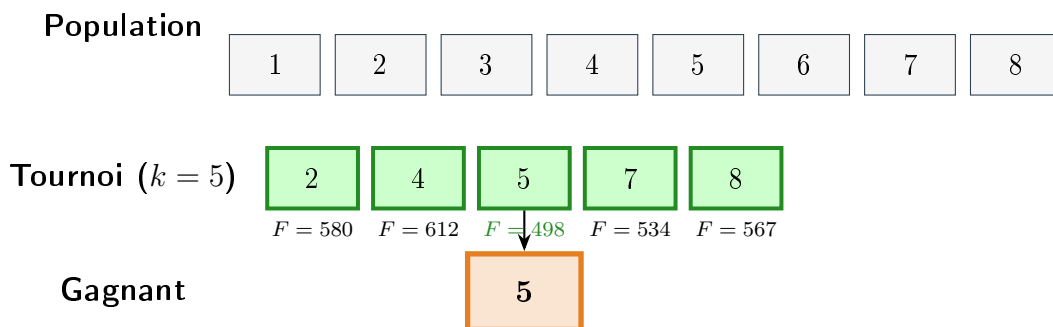


Figure 4: Illustration de la sélection par tournoi avec $k = 5$

```

1 def selection_tournoi(population, taille_tournoi):
2     """
3     Selectionne une solution par tournoi.
4     Le meilleur (score le plus bas) parmi k participants
5     est retourné.
6     """
7     # Choisir des participants aleatoires
8     participants = random.sample(population, taille_tournoi)
9
10    # Trouver le meilleur (score le plus bas = meilleur)
11    meilleur = participants[0]
12    for solution in participants[1:]:
13        if solution['score'] < meilleur['score']:
14            meilleur = solution
15
16    return meilleur

```

Listing 6: Fonction de sélection par tournoi

6.2 Avantages de la Sélection par Tournoi

La sélection par tournoi présente plusieurs avantages significatifs. Premièrement, elle favorise les meilleures solutions tout en donnant une chance aux solutions moins performantes d'être sélectionnées, ce qui maintient la **diversité génétique** de la population. Deuxièmement, la **pression de sélection** peut être facilement ajustée en modifiant la taille du tournoi: un tournoi plus grand exerce une pression plus forte vers les meilleures solutions.

7 Opérateur de Croisement

7.1 Croisement à Deux Points

Le croisement (ou *crossover*) est l'opérateur génétique principal qui permet de combiner les caractéristiques de deux solutions parentes pour créer une nouvelle solution. Pour le TSP, nous utilisons un **croisement à deux points** adapté aux problèmes de permutation.

Procédure du croisement à deux points pour le TSP

1. Choisir deux points de coupure aléatoires **point1** et **point2**
2. Copier le segment entre ces deux points du Parent 2 vers l'Enfant
3. Identifier les villes qui apparaissent en double dans l'Enfant
4. Remplacer les doublons par les villes manquantes

7.2 Illustration du Croisement

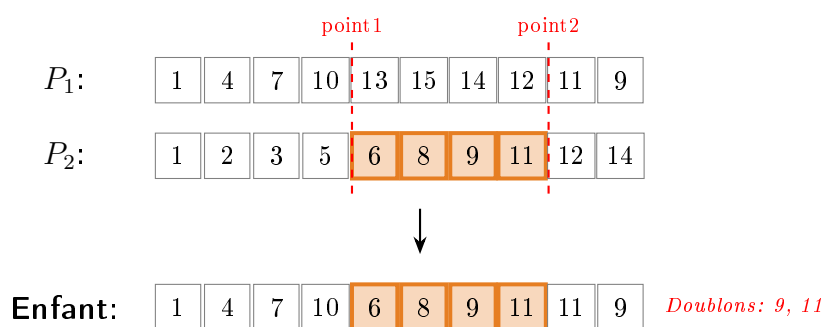


Figure 5: Illustration du croisement à deux points (avant correction des doublons)

7.3 Correction des Doublons

Comme le TSP exige que chaque ville soit visitée exactement une fois, le croisement peut créer des solutions invalides contenant des villes en double. L'algorithme détecte ces doublons et les remplace par les villes manquantes:

```
1 def croisement_deux_points(parent1, parent2, donnees):
2     """
3     Effectue un croisement a deux points entre deux parents.
4     Retourne une nouvelle solution (enfant) issue du croisement.
5     """
6     parcours1 = parent1['parcours']
7     parcours2 = parent2['parcours']
8     taille = len(parcours1)
9
10    # Etape 1: Choisir deux points de coupure aleatoires
11    point1 = random.randint(1, taille - 2)
12    point2 = random.randint(point1 + 1, taille - 1)
13
14    # Etape 2: Commencer avec le parcours du parent1
15    nouveau_parcours = parcours1.copy()
16
17    # Etape 3: Copier le segment du parent2 entre les deux
18    #          points
19    for i in range(point1, point2):
20        nouveau_parcours[i] = parcours2[i]
21
22    # Etape 4: Identifier les doublons
23    villes_vues = set()
24    positions_doublons = []
25
26    for position, ville in enumerate(nouveau_parcours):
27        if ville in villes_vues:
28            positions_doublons.append(position)
29        else:
30            villes_vues.add(ville)
31
32    # Etape 5: Trouver les villes manquantes
33    toutes_villes = set(donnees['villes'])
34    villes_manquantes = list(toutes_villes - villes_vues)
```

```

34     random.shuffle(villes_manquantes)
35
36     # Etape 6: Remplacer les doublons par les villes manquantes
37     for i, position in enumerate(positions_doublons):
38         nouveau_parcours[position] = villes_manquantes[i]
39
40     # Creer et retourner la nouvelle solution
41     enfant = creer_solution_vide()
42     enfant['parcours'] = nouveau_parcours
43     return enfant

```

Listing 7: Fonction de croisement à deux points

8 Opérateur de Mutation

8.1 Mutation par Échange

La mutation est un opérateur génétique qui introduit de la variabilité dans la population en modifiant légèrement certaines solutions. Pour le TSP, nous utilisons une **mutation par échange** (*swap mutation*) qui consiste à échanger la position de deux villes dans le parcours.

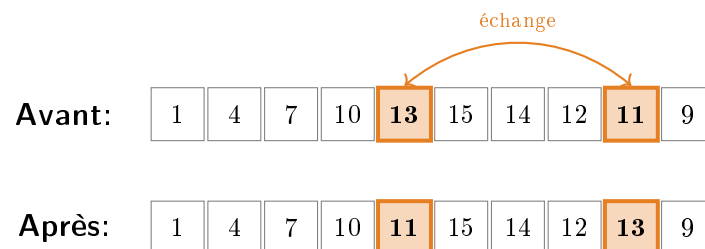


Figure 6: Illustration de la mutation par échange

```

1 def mutation_echange(solution, taux_mutation):
2     """
3     Applique une mutation par echange de deux villes.
4     La mutation n'est appliquee qu'avec une certaine probabilite
5     (definie par le taux de mutation).
6
7     Cette fonction modifie la solution en place.
8     """
9     # Verifier si on effectue la mutation (selon le taux)

```

```
10     if random.random() < taux_mutation:
11         parcours = solution['parcours']
12         taille = len(parcours)
13
14         if taille >= 2:
15             # Choisir deux positions différentes aleatoirement
16             i = random.randint(0, taille - 1)
17             j = random.randint(0, taille - 1)
18
19             # S'assurer que i et j sont différents
20             while j == i:
21                 j = random.randint(0, taille - 1)
22
23             # Echanger les deux villes
24             parcours[i], parcours[j] = parcours[j], parcours[i]
```

Listing 8: Fonction de mutation par échange

8.2 Rôle de la Mutation

La mutation joue un rôle essentiel dans l'algorithme génétique en permettant d'explorer de nouvelles régions de l'espace des solutions. Sans mutation, l'algorithme pourrait converger prématurément vers un optimum local, incapable d'en sortir puisque le croisement ne peut que recombinaison le matériel génétique déjà présent dans la population.

Avec un taux de mutation de 15%, environ une solution sur sept subira une modification, ce qui maintient un bon équilibre entre **exploitation** (amélioration des solutions existantes) et **exploration** (découverte de nouvelles solutions).

9 Algorithme Génétique Principal

9.1 Vue d'Ensemble

L'algorithme génétique suit un processus itératif qui simule l'évolution naturelle. À chaque génération, les meilleures solutions sont sélectionnées pour produire de nouvelles solutions via les opérateurs de croisement et de mutation.

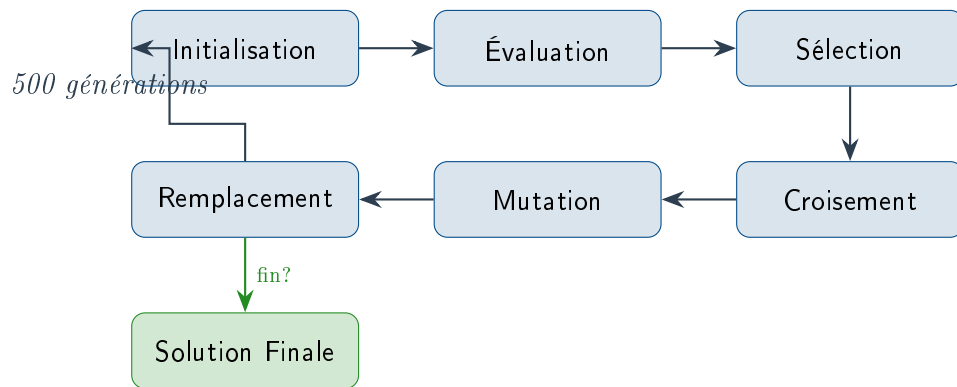


Figure 7: Diagramme du flux de l'algorithme génétique

9.2 Création de la Population Initiale

La première étape consiste à générer une population de solutions aléatoires. Chaque solution est une permutation aléatoire des 15 villes:

```

1 def creer_population_initiale(taille, donnees):
2     """
3     Cree une population initiale de solutions aleatoires.
4     """
5     population = []
6     for _ in range(taille):
7         solution = creer_solution_aleatoire(donnees)
8         population.append(solution)
9     return population

```

Listing 9: Création de la population initiale

9.3 Création d'une Nouvelle Génération

À chaque itération, une nouvelle génération est créée à partir de la population actuelle. Le processus utilise l'**élitisme** (conservation du meilleur individu) combiné avec les opérateurs de sélection, croisement et mutation:

```

1 def creer_nouvelle_generation(population, donnees, params):
2     """
3     Cree la generation suivante a partir de la population
4     actuelle.
5     """
6     nouvelle_population = []

```

```
6
7     # Elitisme: conserver le meilleur individu
8     meilleur = copier_solution(population[0])
9     nouvelle_population.append(meilleur)
10
11     # Remplir le reste de la population
12     while len(nouvelle_population) <
13         params['taille_population']:
14
15         # Decider si on fait un croisement ou une copie
16         if random.random() < params['taux_croisement']:
17             # Croisement entre deux parents
18             parent1 = selection_tournoi(population,
19                                         params['taille_tournoi'])
20             parent2 = selection_tournoi(population,
21                                         params['taille_tournoi'])
22             enfant = croisement_deux_points(parent1, parent2,
23                                             donnees)
24         else:
25             # Copie d'un parent selectionne
26             parent = selection_tournoi(population,
27                                         params['taille_tournoi'])
28             enfant = copier_solution(parent)
29
30         # Appliquer la mutation
31         mutation_echange(enfant, params['taux_mutation'])
32
33     nouvelle_population.append(enfant)
34
35     return nouvelle_population
```

Listing 10: Création d'une nouvelle génération

9.4 Boucle Principale de l'Algorithme

L'algorithme principal orchestre l'ensemble du processus évolutif sur 500 générations:

```
1 def executer_algorithme_genetique(donnees, params):
2     """
3     Execute l'algorithme genetique complet.
```



```
4     Retourne la meilleure solution trouvee.
5     """
6     print(f"Demarrage de l'algorithme genetique")
7     print(f"Population: {params['taille_population']} "
8           f"individus")
9
10    # Etape 1: Creer la population initiale
11    population = creer_population_initiale(
12        params['taille_population'], donnees)
13
14    # Variables pour suivre le meilleur global
15    meilleur_global = None
16    generation_meilleur = 0
17
18    # Etape 2: Boucle principale (500 iterations)
19    for generation in range(params['nb_generations']):
20
21        # Evaluer toutes les solutions
22        evaluer_population(population, donnees)
23
24        # Trier par score (meilleur en premier)
25        trier_population(population)
26
27        # Verifier si on a trouve un meilleur global
28        champion = population[0]
29
30        if (meilleur_global is None or
31            champion['score'] < meilleur_global['score']):
32            meilleur_global = copier_solution(champion)
33            generation_meilleur = generation
34            print(f"Generation {generation:4d}: "
35                  f"F={champion['score']} "
36                  f"(dist={champion['distance']}, "
37                  f"cout={champion['cout']})")
38
39        # Creer la generation suivante
40        population = creer_nouvelle_generation(
41            population, donnees, params)
42
```

```
43     print(f"\nMeilleure solution trouvee "  
44           f"a la generation {generation_meilleur}")  
45  
46     return meilleur_global
```

Listing 11: Fonction principale de l'algorithme génétique

10 Sauvegarde et Export des Résultats

10.1 Format XML de Sortie

Les solutions sont sauvegardées dans un fichier XML structuré qui permet de conserver l'historique des meilleures solutions trouvées:

```
1 <?xml version="1.0" encoding="UTF-8"?>  
2 <TSP_Solutions>  
3     <ProblemType>Symmetric</ProblemType>  
4     <Note>Chaque tour represente une classe d'equivalence  
5         de 30 solutions identiques (15 rotations x 2  
6             directions).  
7         Seule la forme canonique est stockee.</Note>  
8     <BestFitness>512</BestFitness>  
9     <TotalSolutions>2</TotalSolutions>  
10  
11     <Solution id="1">  
12         <Tour>C1,C2,C3,C5,C6,C8,C9,C11,C12,C14,  
13             C15,C13,C10,C7,C4,C1</Tour>  
14         <Distance>204</Distance>  
15         <Cost>308</Cost>  
16     </Solution>  
</TSP_Solutions>
```

Listing 12: Format XML des solutions

Forme canonique uniquement

Pour un TSP symétrique, chaque tour physique possède 30 représentations équivalentes (15 rotations $\times$ 2 directions). Le fichier XML ne stocke que la **forme canonique** de chaque solution, évitant ainsi la redondance. Stocker les 30 représentations serait non seulement un gaspillage d'espace, mais pourrait également induire en erreur le lecteur en lui faisant croire que 30 solutions distinctes ont été trouvées alors qu'il n'y en a qu'une seule.

10.2 Logique de Sauvegarde

L'algorithme implémente une logique de sauvegarde intelligente qui gère plusieurs exécutions successives:

Table 5: Règles de sauvegarde des solutions

Situation	Action
$F_{\text{nouveau}} < F_{\text{précédent}}$	Remplacer toutes les solutions existantes par la nouvelle
$F_{\text{nouveau}} = F_{\text{précédent}}$	Ajouter à la liste si c'est un nouveau parcours (non équivalent)
$F_{\text{nouveau}} > F_{\text{précédent}}$	Conserver les solutions existantes

Cette approche permet de collecter toutes les solutions optimales distinctes, ce qui peut être utile lorsque plusieurs parcours différents atteignent le même score minimal.

11 Conclusion et Perspectives

11.1 Récapitulatif

Cette implémentation d'un algorithme génétique pour le problème du voyageur de commerce bi-objectif illustre les principes fondamentaux des métaheuristiques évolutionnaires. L'algorithme combine efficacement les mécanismes de sélection par tournoi, croisement à deux points et mutation par échange pour explorer l'espace des solutions de manière efficace.

Les points clés de cette implémentation sont:

1. Une **représentation par permutation** adaptée au TSP
2. Un **croisement à deux points** avec correction automatique des doublons
3. Une **stratégie élitiste** préservant la meilleure solution

4. Un système de **sauvegarde XML** permettant de collecter les solutions optimales
5. Une **analyse combinatoire** précise réduisant l'espace de recherche de $n!$ à $\frac{(n-1)!}{2}$ pour le cas symétrique

11.2 Améliorations Possibles

Plusieurs améliorations pourraient être envisagées pour des versions futures de l'algorithme:

- Implémenter d'autres opérateurs de croisement comme le *Order Crossover* (OX) ou le *Partially Mapped Crossover* (PMX)
- Ajouter des mutations plus sophistiquées comme la *2-opt* ou la *3-opt*
- Utiliser une approche Pareto pour le traitement multi-objectif (NSGA-II)
- Implémenter un mécanisme d'auto-adaptation des paramètres
- Paralléliser l'évaluation des solutions pour accélérer le calcul
- Exploiter systématiquement la canonicalisation pour éliminer les doublons de la population